

# Instructor Handout 7: Microprogrammed Control Unit Design

Auqib Hamid Lone

*Department of Computer Science & Engineering, GCET Ganderbal*

August 8, 2026

**Instructor Copy — Not for Student Distribution**

## 7.1 Before You Teach This Lecture

- **Mano, *Computer System Architecture*, 3rd ed., Ch. 7 in full, pp. 213–239** (fully indexed, confirmed-correct citation for this lecture — unlike Week 6’s anchor reading, this one was already correct).

Key sub-sections: 7-1 Control Memory (p. 213); 7-2 Address Sequencing (p. 215–224, the CAR/4-path addressing this lecture’s “mapping unit” simplifies); 7-3 Microprogram Example (p. 219–229, Figs. 7-1, 7-2, and 7-3); 7-4 Design of Control Unit (p. 230–239, Figs. 7-6 and 7-7 — a real 3-decoder F1/F2/F3 realization that is itself a worked horizontal/vertical hybrid, worth reading before Act 3’s encoding-width discussion).

- **Stallings, *Computer Organization and Architecture*, Ch. 16** — *not yet page-indexed*, and the course knowledge base flags an edition mismatch: the supplied PDF is the 11th edition, not the 2015/10th edition the bibliography key implies. If you cross-check a page number against your own physical/PDF copy, confirm which edition it is first — chapter numbering may not match (the 11th edition’s control-unit chapter is numbered differently).

## 7.2 Extra Worked Examples

### 7.2.1 A Different Instruction’s Microprogram

Extend the lecture’s ADD microprogram to  $R7 \leftarrow R0 - R1$ :

| Microaddress | RTL                       | Control word                | Next  |
|--------------|---------------------------|-----------------------------|-------|
| 0            | $Y \leftarrow [R0]$       | $R0_{out}, Y_{in}$          | 1     |
| 1            | $Z \leftarrow [Y] - [R1]$ | $R1_{out}, ALU=sub, Z_{in}$ | 2     |
| 2            | $R7 \leftarrow [Z]$       | $Z_{out}, R7_{in}, END$     | fetch |

**Instructor note:** worth explicitly asking students where this microprogram would need to be *stored* relative to ADD’s (e.g., if ADD occupies rows 0–2, this one must occupy a disjoint range, say rows 3–5) — this previews the mapping-unit question below.

### 7.2.2 Vertical Encoding for a Smaller Signal Set

The lecture’s worked vertical-encoding example groups ADD’s 8 signals into fields. For contrast, consider a hypothetical 2-step LOAD R1, (R2) microprogram using only 4 distinct signals:  $R2_{out}$ ,  $MAR_{in}$ , MemRead, and  $R1_{in}$  (plus END).

With only two mutually-exclusive bus-source options ( $R2_{out}$  or none) and two destination options ( $MAR_{in}$  or  $R1_{in}$ ), a 1-bit source field and a 1-bit destination field suffice, plus 1 bit each for MemRead and END: **4 bits** vertical versus **5 bits** horizontal (one per signal) — a smaller absolute saving than the ADD example, useful for making the point that the vertical-encoding payoff scales *with* the signal count, not independently of it.

## 7.3 Teaching Notes

*Instructional suggestions, not fact-checked content — adapt freely.*

### 7.3.1 Microinstructions and Control Memory

- **Common misconception:** students sometimes conclude microprogrammed control has “no gates at all.” Clarify: it replaces the *custom, per-signal* AND-OR gates of the control matrix with generic memory-decode logic inside CM — gates still exist, they are just not derived and wired by hand per instruction.

### 7.3.2 Address Sequencing

- **Board sequence:** draw MPC and CM first (mirroring how you drew SC/decoder in Week 6, for continuity), then add the mapping unit and next-address feedback *after* asking “how does MPC know where to start for a *different* instruction?” — let the question motivate the mapping unit rather than presenting it as a given.
- **Live question:** “If ADD’s microprogram occupies rows 0–2 and SUB’s is stored right after it, which row should the mapping unit send MPC to for SUB?” (Answer: row 3 — a quick concrete check that mapping is genuinely opcode-specific, not automatic.)

### 7.3.3 Horizontal versus Vertical Encoding

- **Pacing:** the concepts here are simple but the bit-counting arithmetic is where students lose the thread. Do a quick tally on the board (count signals, compute  $\lceil \log_2 n \rceil$  per group) *before* showing the side-by-side diagram, so the diagram confirms arithmetic students already did rather than being their first exposure to it.
- **Common misconception:** students sometimes think vertical encoding is “strictly better” since it’s smaller, missing the decode-delay trade-off entirely. Ask directly: “if vertical is smaller *and* free, why would anyone ever choose horizontal?” and let the answer (no decode delay) surface before you state it.

### 7.3.4 Trade-offs and Synthesis

- **Live question:** tie back to Weeks 5–6 explicitly — “we’ve now seen three separate places a design trades speed for something else (bus count, control-unit style, encoding width). What’s the common pattern?” Good closing discussion prompt before the Week 8 bridge.